# Matrix–Vector Multiplication $y = Ax$: Dense versus CSR Storage

Gustavo Lozano

November 5, 2026

**Abstract**

The product $y = Ax$ is the core operation of iterative solvers and of most first-order optimization methods. This document compares two ways of computing it: with $A$ stored as a dense array and with $A$ stored in compressed sparse row (CSR) format. For each, we give the algorithm, count floating-point operations and memory traffic, derive the arithmetic intensity, estimate run time with the roofline model, find the density at which both formats cost the same, and discuss parallelization with OpenMP. The main conclusion is that both computations are limited by memory bandwidth, so the format that moves fewer bytes wins; for sparse matrices this is CSR by several orders of magnitude.

## Contents

**1 Problem and notation** **2**

**2 Storage formats** **2**
2.1 Dense . . . 2
2.2 Compressed sparse row (CSR) . . . 2
2.3 Running example . . . 2
2.4 Storage comparison for larger matrices . . . 3

**3 Algorithms** **4**
3.1 Dense, row-oriented (dot-product form) . . . 4
3.2 Dense, column-oriented (axpy form) . . . 5
3.3 CSR . . . 5
3.4 Trace on the running example . . . 6

**4 Cost analysis** **6**
4.1 Floating-point operations . . . 6
4.2 Memory traffic . . . 6
4.3 Arithmetic intensity . . . 6

**5 Performance model: the roofline** **7**
5.1 Examples . . . 7

**6 Break-even density** **8**

**7 Memory access patterns** **8**

**8 Parallelization with OpenMP** **9**
8.1 Distributing rows . . . 9
8.2 Memory placement and threads . . . 9
8.3 Vectorization (SIMD) . . . 9

**9 Numerical considerations** **10**

**10 Industry-standard implementations** **10**

**11 Summary** **10**

# 1 Problem and notation

Given $A \in \mathbb{R}^{M \times N}$ and $x \in \mathbb{R}^N$, compute

$$y_i = \sum_{j=0}^{N-1} a_{ij}\, x_j, \qquad i = 0, \ldots, M-1.$$

| Symbol | Meaning |
|---|---|
| $M, N$ | number of rows and columns of $A$ |
| nnz | number of stored nonzero entries of $A$ |
| $n_i$ | number of nonzeros in row $i$, so $\sum_i n_i = \text{nnz}$ |
| $d = \text{nnz}/(MN)$ | density of $A$ |
| $s_v, s_i$ | bytes per value and per index (here $s_v = 8$, double; $s_i = 4$, 32-bit int) |
| $\beta$ | memory bandwidth (bytes/s) |
| $\pi$ | peak floating-point rate (flop/s) |

All indices are 0-based. Costs are given for double-precision values and 32-bit indices.

# 2 Storage formats

## 2.1 Dense

All $MN$ entries are stored, zeros included, in one contiguous array. In *row-major* order (the C convention), $a_{ij}$ is at position $iN + j$; in *column-major* order (the Fortran and BLAS convention), it is at position $i + jM$.

$$\text{memory}_{\text{dense}} = s_v\, MN \text{ bytes.}$$

## 2.2 Compressed sparse row (CSR)

Only the nonzeros are stored, row by row, in three arrays:

- `val`$[0..\text{nnz}-1]$: the nonzero values;
- `col`$[0..\text{nnz}-1]$: the column index of each value;
- `row_ptr`$[0..M]$: row $i$ occupies positions `row_ptr`$[i], \ldots,$ `row_ptr`$[i+1]-1$, and `row_ptr`$[M] =$ nnz.

$$\text{memory}_{\text{CSR}} = (s_v + s_i)\,\text{nnz} + s_i(M+1) = 12\,\text{nnz} + 4(M+1) \text{ bytes.}$$

## 2.3 Running example

$$A = \begin{pmatrix} 10 & 0 & 0 & 0 & 2 & 0 \\ 0 & 3 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 7 & 0 & 0 & 1 \\ 4 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 5 & 0 & 6 \end{pmatrix}, \qquad x = \begin{pmatrix} 1 \\ 2 \\ 3 \\ 4 \\ 5 \\ 6 \end{pmatrix}, \qquad M = N = 6,\ \text{nnz} = 8,\ d = \tfrac{8}{36} \approx 0.22.$$

The exact product is

$$y = Ax = \begin{pmatrix} 10 \cdot 1 + 2 \cdot 5 \\ 3 \cdot 2 \\ 0 \\ 7 \cdot 3 + 1 \cdot 6 \\ 4 \cdot 1 \\ 5 \cdot 4 + 6 \cdot 6 \end{pmatrix} = \begin{pmatrix} 20 \\ 6 \\ 0 \\ 27 \\ 4 \\ 56 \end{pmatrix},$$

which serves as the reference result for testing any implementation. Row 2 is empty, which shows how CSR handles rows with no nonzeros.

$$\begin{aligned} \texttt{val} &= [10, 2, 3, 7, 1, 4, 5, 6], \\ \texttt{col} &= [0, 4, 1, 2, 5, 0, 3, 5], \\ \texttt{row_ptr} &= [0, 2, 3, 3, 5, 6, 8]. \end{aligned}$$

Storage:

| Array | Elements | Bytes |
|---|---|---|
| Dense $A$ ($8MN$) | 36 doubles | 288 |
| CSR `val` (8 nnz) | 8 doubles | 64 |
| CSR `col` (4 nnz) | 8 ints | 32 |
| CSR `row_ptr` ($4(M+1)$) | 7 ints | 28 |
| CSR total | | 124 |

CSR uses 124 bytes instead of 288, a factor of 2.3 less, even for this tiny matrix. Note that the $4(M+1)$ bytes of `row_ptr` do not depend on nnz: for very small matrices this fixed overhead is noticeable, and a small matrix needs a density somewhat below the asymptotic break-even $d^* \approx 2/3$ (section 6) before CSR is smaller.

### 2.4 Storage comparison for larger matrices

For a square matrix ($M = N$) the two formulas are

$$\text{memory}_{\text{dense}} = 8N^2, \qquad \text{memory}_{\text{CSR}} = 12\,\text{nnz} + 4(N+1),$$

and the ratio is

$$\rho = \frac{\text{memory}_{\text{dense}}}{\text{memory}_{\text{CSR}}} = \frac{8N^2}{12\,\text{nnz} + 4(N+1)}.$$

Dense storage grows with $N^2$ no matter how many entries are zero; CSR grows with nnz. Two cases are worth separating.

**Fixed number of nonzeros per row.** Many sparse matrices have about $k$ nonzeros per row whatever their size (for example $k \approx 5$ for a 2-D finite-difference grid, or a few variables per constraint in a large optimization model). Then $\text{nnz} = kN$, CSR grows only linearly, and

$$\rho \approx \frac{8N}{12k + 4},$$

so the saving *grows with the size of the matrix*. For $k = 5$, $\rho \approx N/8$:

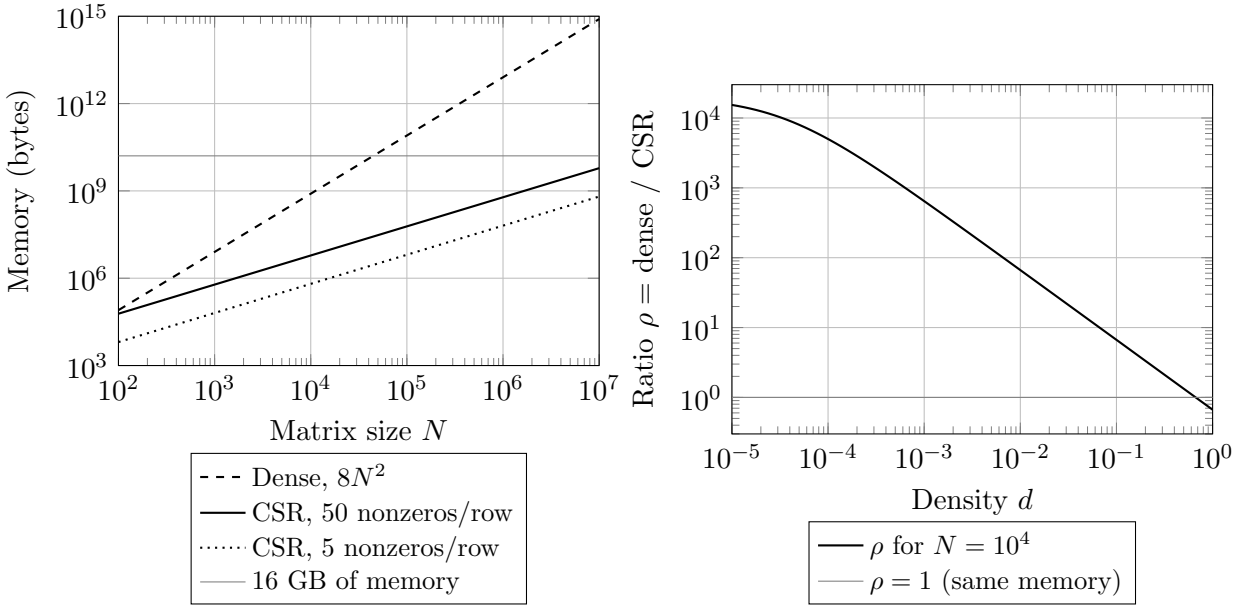
| $N$ | $\text{nnz} = 5N$ | dense | CSR | ratio $\rho$ |
|---|---|---|---|---|
| $10^2$ | $5 \cdot 10^2$ | 80 KB | 6.4 KB | 12.5 |
| $10^3$ | $5 \cdot 10^3$ | 8 MB | 64 KB | 125 |
| $10^4$ | $5 \cdot 10^4$ | 800 MB | 640 KB | 1 250 |
| $10^5$ | $5 \cdot 10^5$ | 80 GB | 6.4 MB | 12 500 |
| $10^6$ | $5 \cdot 10^6$ | 8 TB | 64 MB | 125 000 |

Beyond $N \approx 4.5 \cdot 10^4$ the dense matrix no longer fits in 16 GB of memory, while the CSR matrix still takes only megabytes.

**Fixed size, varying density.** For $N = 10^4$ the dense matrix always takes 800 MB, and CSR depends only on $\text{nnz} = dN^2$:

| density $d$ | nnz | dense | CSR | ratio $\rho$ |
|---|---|---|---|---|
| $10^{-4}$ | $10^4$ | 800 MB | 0.16 MB | 5 000 |
| $10^{-3}$ | $10^5$ | 800 MB | 1.24 MB | 645 |
| $10^{-2}$ | $10^6$ | 800 MB | 12.0 MB | 66 |
| $10^{-1}$ | $10^7$ | 800 MB | 120 MB | 6.7 |
| 0.5 | $5 \cdot 10^7$ | 800 MB | 600 MB | 1.33 |
| 2/3 | $6.7 \cdot 10^7$ | 800 MB | 800 MB | 1.00 |
| 1 | $10^8$ | 800 MB | 1 200 MB | 0.67 |

The storage break-even is at $d \approx 2/3$: above it, the 4-byte index stored with every value makes CSR *larger* than dense.




(a) Memory against size, with $\text{nnz} = kN$.

(b) Memory ratio against density, $N = 10^4$.

Figure 1: Storage of dense and CSR matrices. (a) With a fixed number of nonzeros per row, dense memory grows as $N^2$ and crosses 16 GB near $N = 4.5 \cdot 10^4$, while CSR grows linearly. (b) For a fixed size, CSR saves memory as long as $d < 2/3$; the saving is a factor of about $2/(3d)$ for small $d$.

# 3 Algorithms

## 3.1 Dense, row-oriented (dot-product form)

Natural for row-major storage: each $y_i$ is the dot product of row $i$ with $x$. The inner loop reads $A$ and $x$ contiguously.

**Algorithm 1** Dense $y = Ax$, row-major storage

**Require:** $A[0..MN-1]$ row-major, $x[0..N-1]$
**Ensure:** $y[0..M-1]$
1: **for** $i \leftarrow 0$ **to** $M-1$ **do**
2: $\quad s \leftarrow 0$
3: $\quad$ **for** $j \leftarrow 0$ **to** $N-1$ **do**
4: $\quad\quad s \leftarrow s + A[iN+j] \cdot x[j]$
5: $\quad$ **end for**
6: $\quad y[i] \leftarrow s$
7: **end for**

### 3.2 Dense, column-oriented (axpy form)

Natural for column-major storage: $y$ is built as a linear combination of the columns of $A$, $y = \sum_j x_j\, a_{:,j}$. Each step is an *axpy* ($y \leftarrow y + \alpha a$).

**Algorithm 2** Dense $y = Ax$, column-major storage

**Require:** $A[0..MN-1]$ column-major, $x[0..N-1]$
**Ensure:** $y[0..M-1]$
1: **for** $i \leftarrow 0$ **to** $M-1$ **do**
2: $\quad y[i] \leftarrow 0$
3: **end for**
4: **for** $j \leftarrow 0$ **to** $N-1$ **do**
5: $\quad \alpha \leftarrow x[j]$
6: $\quad$ **for** $i \leftarrow 0$ **to** $M-1$ **do**
7: $\quad\quad y[i] \leftarrow y[i] + A[i+jM] \cdot \alpha$
8: $\quad$ **end for**
9: **end for**

Both forms do the same arithmetic. What matters is that the inner loop walks through $A$ in the order it is stored; using the wrong loop order for the storage makes every access to $A$ jump by $N$ (or $M$) elements and destroys cache efficiency.

### 3.3 CSR

The same dot-product form as algorithm 1, but the inner loop visits only the $n_i$ nonzeros of row $i$, and reads $x$ through the column index.

**Algorithm 3** CSR $y = Ax$

**Require:** `row_ptr`$[0..M]$, `col`$[0..\text{nnz}-1]$, `val`$[0..\text{nnz}-1]$, $x[0..N-1]$
**Ensure:** $y[0..M-1]$
1: **for** $i \leftarrow 0$ **to** $M-1$ **do**
2: $\quad s \leftarrow 0$
3: $\quad$ **for** $k \leftarrow$ `row_ptr`$[i]$ **to** `row_ptr`$[i+1]-1$ **do**
4: $\quad\quad s \leftarrow s +$ `val`$[k] \cdot x[$`col`$[k]]$ ▷ indirect access to $x$
5: $\quad$ **end for**
6: $\quad y[i] \leftarrow s$
7: **end for**

An empty row (`row_ptr`$[i] =$ `row_ptr`$[i+1]$) skips the inner loop and correctly produces $y_i = 0$.

### 3.4 Trace on the running example

| $i$ | $k$ range | terms | $y_i$ |
|---|---|---|---|
| 0 | 0–1 | $10 \cdot x_0 + 2 \cdot x_4 = 10 + 10$ | 20 |
| 1 | 2 | $3 \cdot x_1$ | 6 |
| 2 | none (3 to 2) | empty row | 0 |
| 3 | 3–4 | $7 \cdot x_2 + 1 \cdot x_5 = 21 + 6$ | 27 |
| 4 | 5 | $4 \cdot x_0$ | 4 |
| 5 | 6–7 | $5 \cdot x_3 + 6 \cdot x_5 = 20 + 36$ | 56 |

So $y = (20, 6, 0, 27, 4, 56)^\mathsf{T}$. The dense algorithm performs $2MN = 72$ floating-point operations, 56 of them on zeros; CSR performs $2\,\mathrm{nnz} = 16$, all useful.

## 4 Cost analysis

### 4.1 Floating-point operations

Each stored entry contributes one multiplication and one addition:

$$W_{\text{dense}} = 2MN, \qquad W_{\text{CSR}} = 2\,\mathrm{nnz} = 2\,d\,MN.$$

CSR does a fraction $d$ of the dense work.

### 4.2 Memory traffic

Each entry of $A$ is used exactly once, so $A$ must always be streamed from main memory (for matrices larger than the cache). The vectors behave differently:

- $y$: written once, $s_v M$ bytes.
- $x$, dense: read contiguously and reused by every row; if it stays in cache it costs $s_v N$ bytes in total.
- $x$, CSR: read through `col`$[k]$. In the *best case* (good locality, $x$ stays in cache) it costs $s_v N$ bytes; in the *worst case* every access misses cache and it costs about $s_v\,\mathrm{nnz}$ bytes (even more if each miss loads a whole 64-byte cache line for one useful value).

| Data | Dense (bytes) | CSR (bytes) |
|---|---|---|
| Matrix values | $8MN$ | $8\,\mathrm{nnz}$ |
| Index arrays | – | $4\,\mathrm{nnz} + 4(M+1)$ |
| $x$ (best / worst) | $8N$ | $8N$ / $8\,\mathrm{nnz}$ |
| $y$ | $8M$ | $8M$ |
| Total, best case | $8MN + 8M + 8N$ | $12\,\mathrm{nnz} + 12M + 8N + 4$ |
| Total, worst case | – | $20\,\mathrm{nnz} + 12M + 4$ |

### 4.3 Arithmetic intensity

The arithmetic intensity $I = W/Q$ (flops per byte moved) decides whether a computation is limited by the processor or by memory. Keeping only the dominant terms:

$$I_{\text{dense}} \approx \frac{2MN}{8MN} = \frac{1}{4}, \qquad I_{\text{CSR}}^{\text{best}} \approx \frac{2\,\mathrm{nnz}}{12\,\mathrm{nnz}} = \frac{1}{6}, \qquad I_{\text{CSR}}^{\text{worst}} \approx \frac{2\,\mathrm{nnz}}{20\,\mathrm{nnz}} = \frac{1}{10}.$$

Every value read from $A$ is used for exactly two flops, and there is no way to reuse it: unlike matrix–matrix multiplication, $y = Ax$ cannot be restructured to raise its intensity. CSR has a slightly *lower* intensity than dense because each value carries a 4-byte index.

# 5 Performance model: the roofline

The roofline model bounds the attainable rate by

$$P = \min(\pi,\ I\,\beta).$$

Modern processors have $\pi/\beta$ (the *ridge point*) between about 5 and 20 flop/byte. Since $I \le 1/4$ for both formats, $y = Ax$ is *memory-bound*: the run time is essentially

$$T \approx \frac{Q}{\beta}.$$

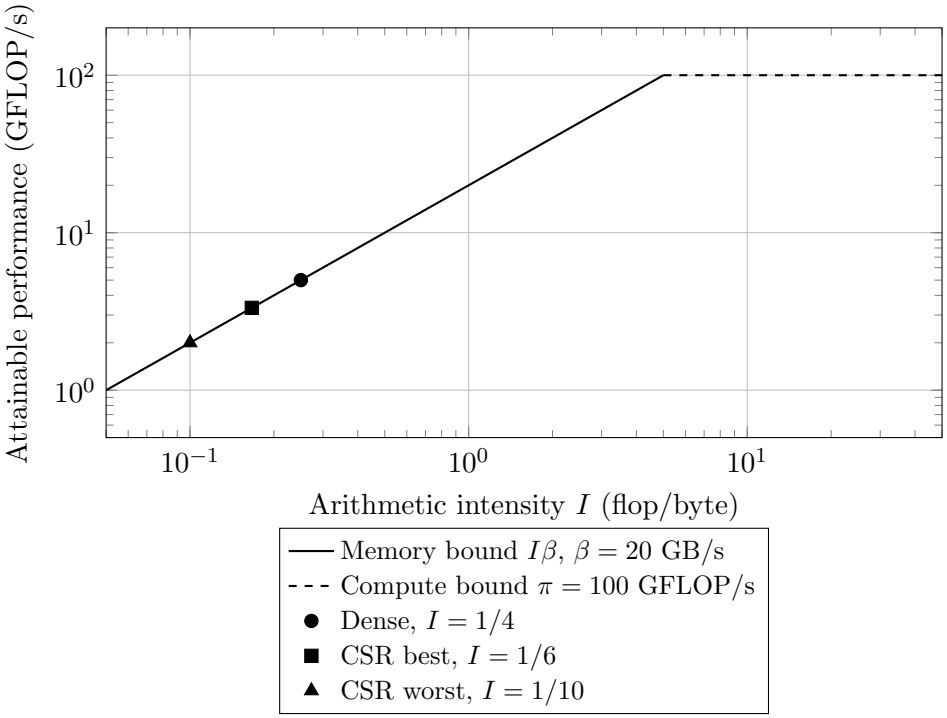



Figure 2: Roofline for an illustrative machine ($\beta = 20$ GB/s, $\pi = 100$ GFLOP/s). Both formats sit far left of the ridge point ($I = 5$): they use at most 5% of the peak floating-point rate.

The *rate* of CSR (flop/s) is a little lower than that of dense, but CSR does a factor $d$ fewer flops, so its *time* is far smaller.

## 5.1 Examples

All times use $T \approx Q/\beta$ with $\beta = 20$ GB/s and the best-case CSR traffic.

| Case | Dense | | CSR | |
|---|---|---|---|---|
| | traffic | time | traffic | time |
| $N = 10^4$, $d = 10^{-2}$ (nnz $= 10^6$) | 800 MB | 40 ms | 12.2 MB | 0.6 ms |
| $N = 10^6$, nnz $= 5 \cdot 10^6$ | 8 TB | 400 s | 80 MB | 4.0 ms |

In the second case the dense matrix does not even fit in the memory of a workstation; in the first, CSR is about 65 times faster.

## 6 Break-even density

Setting the best-case CSR traffic equal to the dense traffic and keeping dominant terms,

$$12\,\mathrm{nnz} \approx 8MN \quad \Longrightarrow \quad d^*_{\text{best}} \approx \frac{2}{3}, \qquad \text{and with worst-case } x \text{ traffic,} \qquad d^*_{\text{worst}} \approx \frac{8}{20} = 0.4.$$

Below $d^*$, CSR moves fewer bytes and is faster in this model.

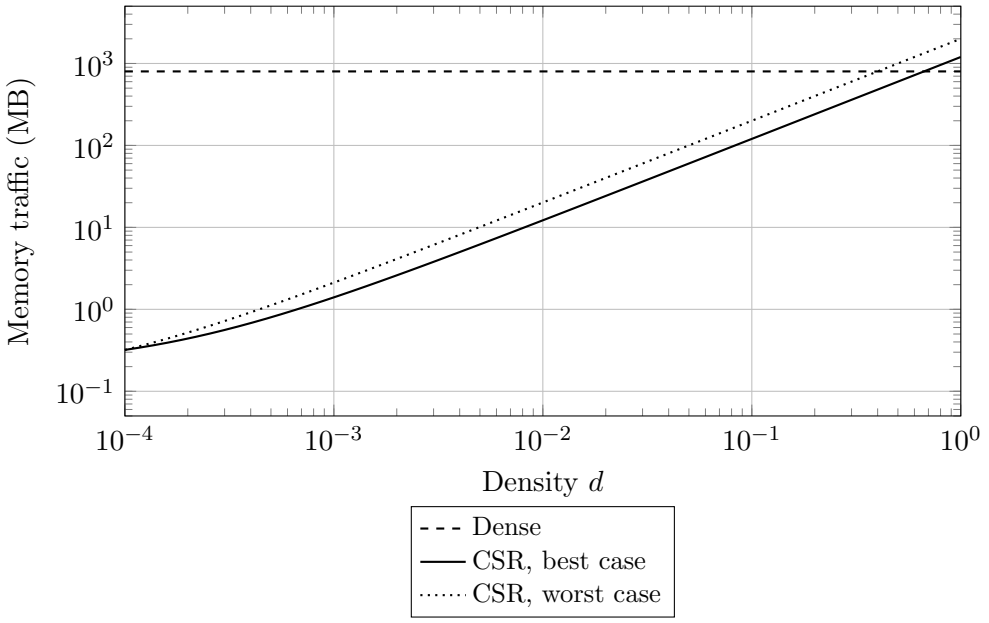



Figure 3: Memory traffic of one product $y = Ax$ for $M = N = 10^4$. The curves cross at $d \approx 0.67$ (best case) and $d \approx 0.4$ (worst case).

In practice the crossover is lower than the model predicts, because the dense inner loop vectorizes perfectly with SIMD instructions and is prefetched by hardware, while the CSR loop has an indirect access, a short trip count for short rows, and loop overhead per row. A common rule of thumb is that CSR pays off below a few percent density; the exact point depends on the machine and the sparsity pattern and should be measured.

## 7 Memory access patterns

| Access | Dense (row-major) | CSR |
|---|---|---|
| $A$ | contiguous stream | contiguous stream (`val`, `col`) |
| $x$ | contiguous, reused per row | *gather*: $x$[`col`[$k$]] |
| $y$ | one write per row | one write per row |
| Inner loop length | $N$ (long, uniform) | $n_i$ (short, varies) |

The cost of the gather depends on the sparsity pattern:

- **Banded or clustered columns** (e.g. discretized PDEs, nearby columns in each row): consecutive accesses hit the same cache lines, so traffic is close to the best case.
- **Scattered columns** (e.g. random graphs, many optimization models): accesses miss cache and traffic approaches the worst case. Reordering rows and columns (for example with reverse Cuthill–McKee) can bring nonzeros closer to the diagonal and improve locality.
- **Very short rows** (1–3 nonzeros): the per-row overhead (reading `row_ptr`, starting and ending the loop, writing $y_i$) becomes comparable to the useful work.

# 8 Parallelization with OpenMP

## 8.1 Distributing rows

In both formats the rows are independent: each $y_i$ is written by exactly one iteration, so the outer loop can be parallelized without synchronization.

**Algorithm 4** Parallel CSR $y = Ax$ (one row per iteration)

```
1: for i ← 0 to M − 1 in parallel do                    ▷ e.g. omp parallel for
2:     s ← 0
3:     for k ← row_ptr[i] to row_ptr[i + 1] − 1 do
4:         s ← s + val[k] · x[col[k]]
5:     end for
6:     y[i] ← s
7: end for
```

For dense matrices every row costs the same, so a static schedule balances the load perfectly. For CSR, rows can have very different lengths $n_i$. Two common remedies:

1. a dynamic or guided schedule, which hands out chunks of rows as threads become free (simple, some scheduling overhead);
2. an nnz-*balanced static partition*: give each thread a contiguous block of rows containing about nnz/$T$ nonzeros (algorithm 5). This keeps the low overhead of a static schedule and the memory locality of contiguous blocks.

**Algorithm 5** nnz-balanced row partition for $T$ threads

**Require:** `row_ptr`$[0..M]$, number of threads $T$
**Ensure:** `start`$[0..T]$: thread $t$ processes rows `start`$[t], \ldots,$ `start`$[t+1]-1$

```
1: start[0] ← 0,    start[T] ← M
2: for t ← 1 to T − 1 do
3:     τ ← ⌊t · nnz/T⌋
4:     start[t] ← smallest i with row_ptr[i] ≥ τ        ▷ binary search, O(log M)
5: end for
```

A single very long row still cannot be split by this scheme; merge-based SpMV [5] solves that case by splitting the work at the level of individual nonzeros.

## 8.2 Memory placement and threads

Because the operation is memory-bound, adding threads helps only until the memory bandwidth is saturated, which often happens before all cores are in use. On machines with several memory controllers (NUMA), data should be initialized by the same threads, with the same schedule, that later use it (*first-touch* placement), and threads should be pinned to cores (`OMP_PROC_BIND`, `OMP_PLACES`) [8].

## 8.3 Vectorization (SIMD)

The dense inner loop reads contiguous data and vectorizes directly (for example with `omp simd reduction(+:s)`). The CSR inner loop needs a *gather* instruction to load $x[\texttt{col}[k]]$, which is slower than a contiguous load, and its short, variable trip count leaves vector lanes unused. Formats such as SELL-$C$-$\sigma$ [6] rearrange the nonzeros into fixed-width chunks to improve SIMD and GPU efficiency.

## 9 Numerical considerations

Both algorithms compute the same sums, but floating-point addition is not associative. Dense algorithms add the zero terms (which does not change the result) and may sum in a different order when vectorized or blocked; parallel or SIMD versions of either format may also change the order. Results can therefore differ in the last bits between implementations and between runs with different numbers of threads. The rounding error bound is the same for both: $|\hat{y}_i - y_i| \le \gamma_n \sum_j |a_{ij}|\,|x_j|$ with $\gamma_n = nu/(1-nu)$, where $u$ is the unit roundoff and $n$ is the number of terms summed ($N$ for dense, $n_i$ for CSR), so CSR's bound is never worse.

## 10 Industry-standard implementations

| Format | Standard routines |
|---|---|
| Dense | BLAS level 2 `dgemv` (OpenBLAS, BLIS, Intel MKL); `cblas_dgemv` accepts row- or column-major layout |
| CSR | Intel MKL Sparse BLAS (`mkl_sparse_d_mv`); NVIDIA cuSPARSE (`cusparseSpMV`); PETSc (`MatMult`) |

There is no single reference implementation of sparse matrix–vector multiplication comparable to the reference BLAS for dense operations; each library uses its own data structures and tuning. A custom implementation is best validated against one of these libraries, both for correctness and for speed.

## 11 Summary

| | Dense | CSR |
|---|---|---|
| Storage | $8MN$ | $12\,\text{nnz} + 4(M+1)$ |
| Flops | $2MN$ | $2\,\text{nnz}$ |
| Memory traffic | $\approx 8MN$ | $\approx 12\,\text{nnz}$ to $20\,\text{nnz}$ |
| Arithmetic intensity | $1/4$ | $1/6$ to $1/10$ |
| Bound by | memory bandwidth | memory bandwidth |
| Access to $x$ | contiguous | indirect (gather) |
| Parallel load balance | automatic | needs care if $n_i$ vary |
| SIMD | excellent | limited by gather |
| Faster when | $d$ is large (roughly $> 0.4$ in the model) | $d$ is small |

**Conclusions.**

1. $y = Ax$ is memory-bound in both formats; its run time is the number of bytes moved divided by the memory bandwidth.
2. CSR moves about $12\,\text{nnz}$ bytes instead of $8MN$, so for typical sparse matrices ($d \ll 1$) it is faster by roughly a factor $2/(3d)$, and it makes problems feasible that do not fit in memory as dense arrays.
3. The quality of a CSR implementation depends mostly on the access pattern to $x$ and on load balance across threads, not on the arithmetic.

## References

[1] Y. Saad. *Iterative Methods for Sparse Linear Systems*, 2nd ed. SIAM, Philadelphia, 2003.

[2] G. H. Golub and C. F. Van Loan. *Matrix Computations*, 4th ed. Johns Hopkins University Press, Baltimore, 2013.

[3] S. Williams, A. Waterman, and D. Patterson. Roofline: an insightful visual performance model for multicore architectures. *Communications of the ACM*, 52(4):65–76, 2009.

[4] S. Williams, L. Oliker, R. Vuduc, J. Shalf, K. Yelick, and J. Demmel. Optimization of sparse matrix–vector multiplication on emerging multicore platforms. *Parallel Computing*, 35(3):178–194, 2009.

[5] D. Merrill and M. Garland. Merge-based parallel sparse matrix–vector multiplication. In *Proceedings of SC16*, 2016.

[6] M. Kreutzer, G. Hager, G. Wellein, H. Fehske, and A. R. Bishop. A unified sparse matrix data format for efficient general sparse matrix–vector multiplication on modern processors with wide SIMD units. *SIAM Journal on Scientific Computing*, 36(5):C401–C423, 2014.

[7] B. Chapman, G. Jost, and R. van der Pas. *Using OpenMP: Portable Shared Memory Parallel Programming*. MIT Press, Cambridge, MA, 2007.

[8] R. van der Pas, E. Stotzer, and C. Terboven. *Using OpenMP – The Next Step: Affinity, Accelerators, Tasking, and SIMD*. MIT Press, Cambridge, MA, 2017.

[9] R. F. Boisvert, R. Pozo, and K. A. Remington. The Matrix Market exchange formats: initial design. NISTIR 5935, National Institute of Standards and Technology, 1996.